

Chapter 5. Instruction Set Architecture

Addressing Methods

Immediate Addressing

Direct Addressing

Indirect Addressing

Index Addressing

Relative Addressing

Inherent Addressing

Register Transfer Language

Instruction Set Architecture

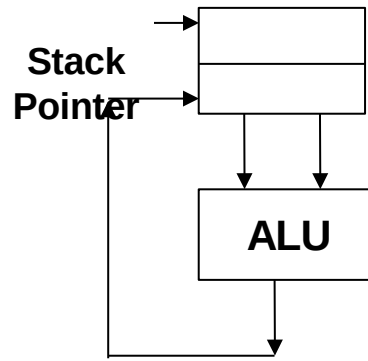
The CPU processes the instructions. Instructions define the tasks the CPU must do. An instruction contains operation information (opcode) and address information. The data at the specified address is called operand. The operand can be in memory or in the register.

A processor's instruction set is designed based on where the operand will be retrieved. Generally, three types of architecture stand out;

- 1. Stack architecture**
- 2. Accumulator architecture**
- 3. General purpose register architecture**

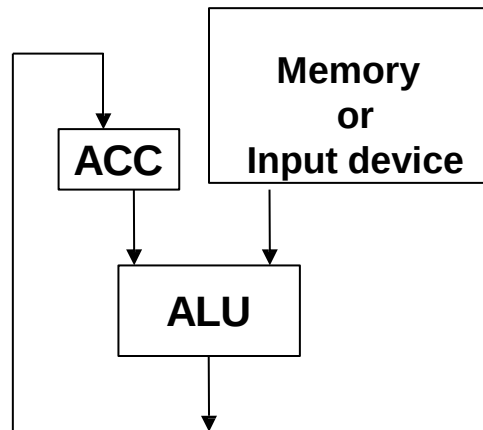
Stack Architecture

The operations are performed on the upper elements of the stack and the result is thrown back onto the stack. In this architecture, transferring information to the stack is done with **push** commands and retrieving information from the stack is done with **pop** commands.



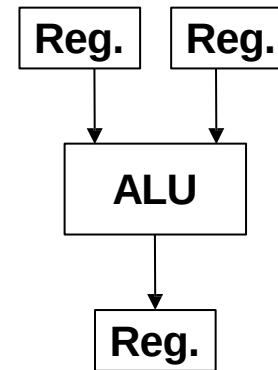
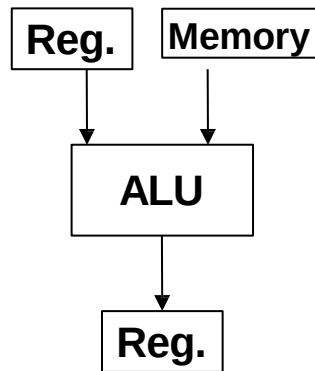
Accumulator Architecture

One of the operands is in memory and the other is in Accumulator (ACC). The process is performed and the result is transferred to the ACC. The ACC is not specified separately in the commands (it is implied). We will use this architecture in our basic computer system that we will design in the future.



General Purpose Register Architecture

In this architecture, operands are specified as registers or memory. It can be of register-memory or register-register type. After operations are performed on the operands, the results are transferred to the registers.



Example: Let's say we want to add two numbers (number_1 and number_2) in memory and put the result (sum) back into memory.

Stack architecture,

Push number _1

Push number _2

Add

Pop total

ACC architecture,

Load number_1

add number _2

store total

Register-memory architecture,

load Reg_1, number _1

Add Reg_2, Reg_1, number _2

Store Reg_2, total

Register-register architecture,

load Reg_1, number _1

load Reg_2, number _2

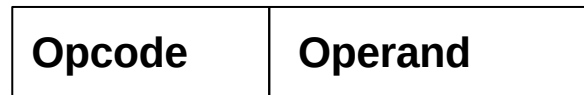
Add Reg_3, Reg_1, Reg_2

store Reg_3, total

Addressing Methods

It was mentioned that a command consists of opcode and address parts. The address part of the command indicates the operand to be operated on. If the operand is in memory, the address of the operand is called the effective address.

1. Immediate Addressing: In this addressing mode, the operand is part of the instruction. In other words, operand is the data in the address part of the command.

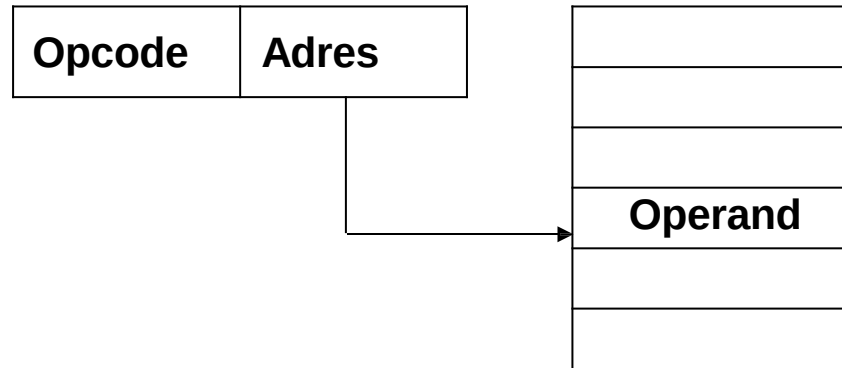


Example: In the Add #3 command, '3' is the operand. This command adds the value 3 to the contents of the ACC and writes the result to the ACC. It is a quickly executed command that does not require access to memory. The # symbol is used to represent the operand itself.

This addressing method can also be used to assign a fixed initial value to variables in high-level languages. (x=3)

Addressing Methods

2. Direct Addressing: In this addressing method, the address part of the instruction indicates the address of the operand.

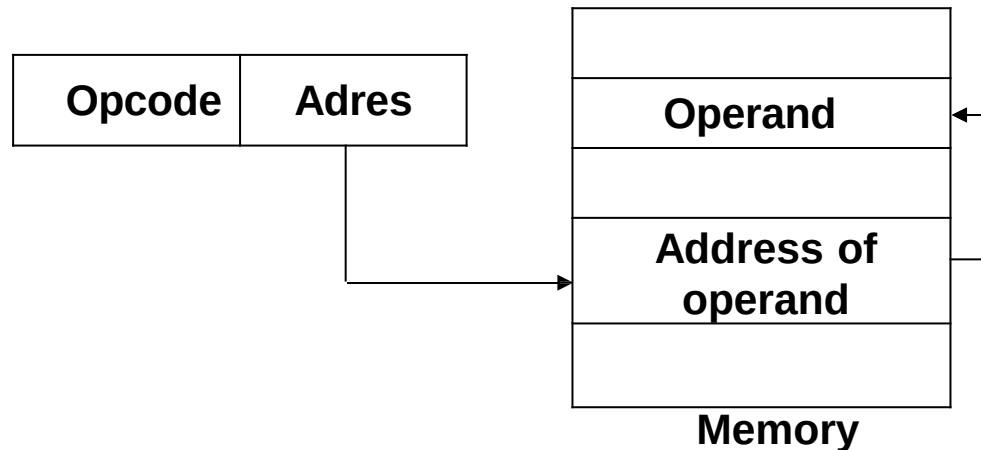


Example: *Add 100* command adds the value stored in memory region 100 to the contents of the ACC and writes the result to the ACC.

- Since memory is referenced in this addressing method, the execution time of instructions is longer than in the immediate mode.
- It is possible to access a limited address space (as many as the number of bits contained in the address part).

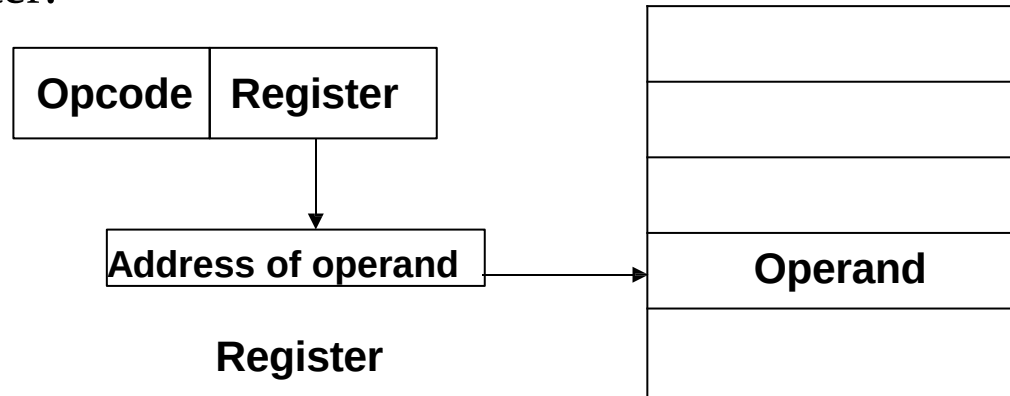
Addressing Methods

3. Indirect Addressing: In this addressing method, the address part of the instruction does not show the address of the operand; It indicates a register or memory area where the effective address of the operand is kept.



Example: *Add (100)* command adds the value of the operand whose address is in memory region 100 to the contents of the ACC and writes the result to the ACC. '()' symbols are used to indicate indirect addressing.

Register - in indirect addressing mode, the effective address of the operand is kept in the register.



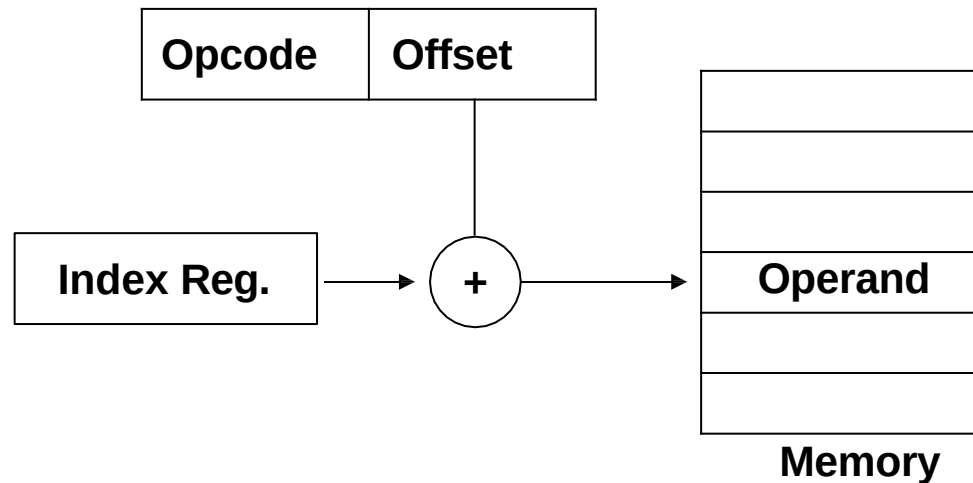
Example: *Add (R1)*

Briefly;

- The execution time of instructions using this addressing method is longer than direct addressing.
- It requires an extra effective address range to access the operand.
- It is possible to expand the address space.

Addressing Methods

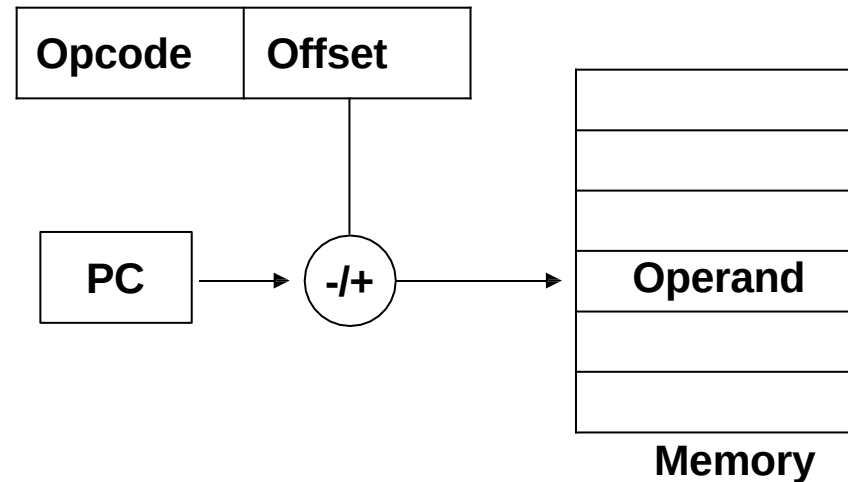
4. Index Addressing: In this addressing mode, an index register is used. It can be used especially if there is sequential access to the elements of an array. The starting address of the directory to be accessed is assigned to the index register. The offset value of the desired array element is located in the operand part of the command.



Example: main {
 k=A[2];
}

Addressing Methods

5. Relative Addressing: It branches somewhere near the current value of the program counter (PC).



Example:

```
main{  
    if (i==1) i++;  
    else i--;  
}
```

Addressing Methods

6. Inherent Addressing: It is an addressing mode that does not contain operand information.

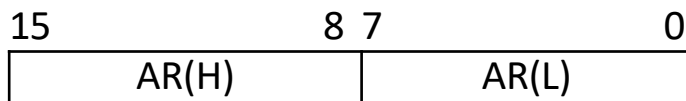
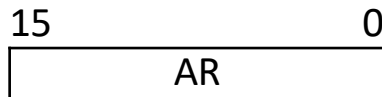
Example: Clr (Cleans the contents of the ACC.)

Inc (Increases ACC content by 1)

Register Transfer Language

The most basic operation in a computer system is information transfer between registers. This language fully expresses the hardware structure and behavior of the computer system. It is also known as Hardware Description Language (HDL).

- Registers are usually expressed with capital letters. They may also contain number information after the letters. Flip flops in an n-bit register are numbered from 0 to (n-1). The registers in our basic computer system are 8 or 16 bits.



- High significance part AR(8-15) or AR(H)
- Low significance AR(0-7) or AR(L)
- A certain group of bits, AR(0-3) or AR₀₋₃

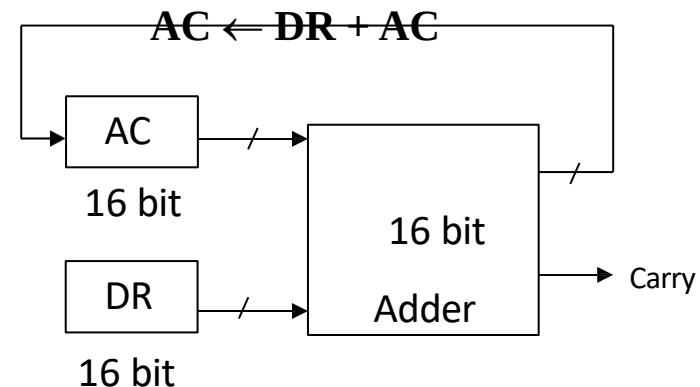
Register Transfer Language

- Register transfer process is indicated by the symbol ' $\leftarrow$ ' and indicates the direction of transfer.

For example, the expression $AR \leftarrow PC$ indicates that the content of the PC will be transferred to AR (It should be noted that the bit numbers must be compatible). As a result of this transfer, the contents of the PC are preserved.

- The results of arithmetic or logic operations can also be transferred to registers;

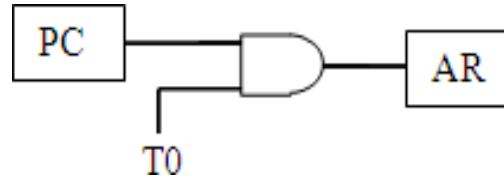
$AC \leftarrow DR + AC$	Addition
$AC \leftarrow DR - AC$	Subtraction
$AC \leftarrow AC'$	Complement
$AC \leftarrow AC' + 1$	2's complement
$AC \leftarrow AC \cap DR$	AND
$AC \leftarrow AC \cup DR$	OR
$AC \leftarrow SHR(AC)$	Shifting right
$AC \leftarrow SHL(AC)$	Shifting left



Register Transfer Language

- If the transfer process will be done with a control signal, this condition is expressed as “Control Signal:”.

Example T0 : $AR \leftarrow PC$



- If more than one transfer (micro-transaction) is to be carried out simultaneously depending on a certain control signal, it is indicated with “,”;

Example X: $AR \leftarrow PC, PC \leftarrow PC + 1$

Register Transfer Language

If different assignments are to be made depending on the status of the X control signal (if-else structure), the following notation is used;

$X : PC \leftarrow AR$

$X' : PC \leftarrow TR$

